

Numerical Analysis chapter 2 programming report

Zhouyue Qian 3220104074 *

(Information and Computational Science Class 2201), Zhejiang University

Due time: December 17, 2024

Abstract

This is the programming assignment for Chapter 2, where I designed an interpolation base class with two extended interpolation methods: Newton and Hermite.

Statement

This report is translated from Chinese to English by the author and AI. [2]

Design

I designed an interpolation base class that includes basic interpolation functions: adding points, deleting points, interpolation, and checking the range of interpolation points. [3] However, I did not incorporate the divided difference table into the base class because not all interpolations have this feature. The base class is inherited by Newton interpolation and Hermite interpolation. [1]

```
class Interpolation
{
protected:
    vector<double> x_values;
    vector<double> y_values;

public:
    virtual void addPoint(double x, double y);
    virtual void removePoint(int index);
    virtual void printPoints(ofstream& file) const; // 在file里输出数据点
    virtual void printPoints(ostream& os) const; // 输出数据点
    virtual double interpolate(double x) = 0;
    virtual void isInrange(double x) const; // 检查x是否在插值范围内
    virtual ~Interpolation() = default;
    virtual void displayPolynomialPoint(ofstream& file, int n) = 0; // 输出n个插值结果的坐标
};
```

*Electronic address: 3220104074@edu.zju.cn

Since Hermite interpolation requires the input of first-order derivative values, I have refactored the operation of interpolation points in Hermite interpolation.

```
void addPoint(double x, double y, double y_prime); // 新的添加点函数
void removePoint(int index); // 新的删除点函数
void printPoints(ostream& os) const override; // 输出输入数据点函数
void printPoints(ofstream& file) const override; // 输出输入数据点函数
```

B

It is about Runge phenomenon. The result is shown in the following figure.

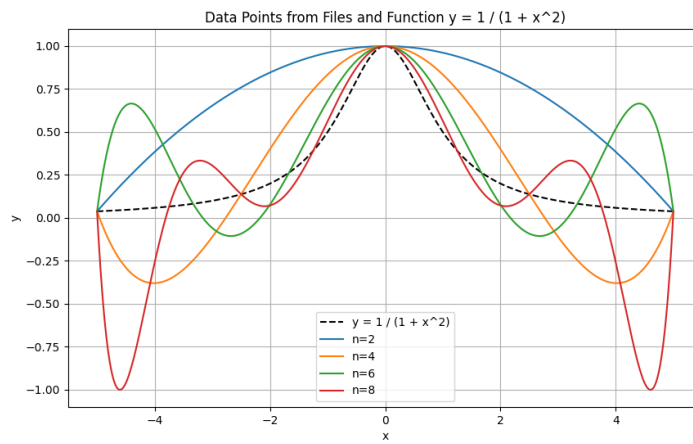


Figure 1: Runge phenomenon

C

It is about Chebyshev nodes. The result is shown in the following figure.

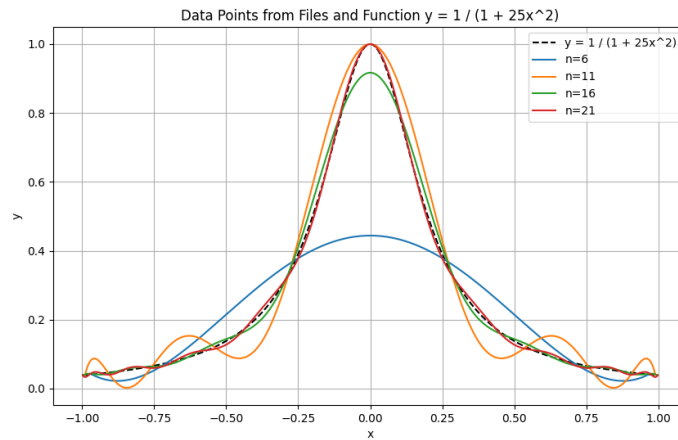


Figure 2: Chebyshev nodes

D

This section is about Hermite interpolation. As the question asks for the first-order derivative, I used the library in scipy to calculate the first-order derivative. The result is shown in the following figure.

```
Polynomial value at t = 10: 742.50041000000004
Derivative value at t = 10: 48.3781190000000055
Derivative exceeds 81.
```

Figure 3: Hermite interpolation

The graph of the function and the graph of its first derivative are shown below.

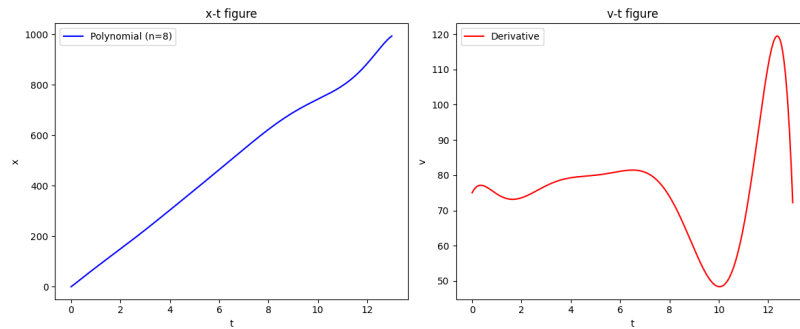


Figure 4: Hermite interpolation

E

This section is about interpolation method's ability to predict the future.

Question 1

The result for the first question is shown in the following figure.

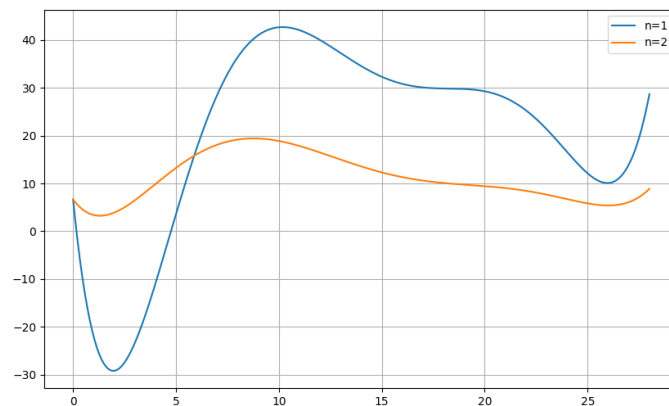


Figure 5: Interpolation

We could find that there was a period of time when the weight was a negative number, which is impossible.

Question 2

As we all know, the interpolation method is not suitable for predicting the future. This is the error term of interpolation:

$$R_n(f;x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

From the error term of interpolation, it can be seen that when the interpolation point is not within the range of the interpolation points, the error will be very large. Therefore, it is not reasonable to directly interpolate to see the weight 15 days later.

F

This section is about the Bezier curve.

Design

I designed a Bezier curve class and a curve class. In curve class, I defined the basic functions of a curve: get points and tangents.

```
class Curve {
protected:
    std::vector<Point> points;
    std::vector<Point> tangents;
public:
    Curve(const std::vector<Point>& pts, const std::vector<Point>& tangs): points(pts), tangents(tangs)
    {}
    // 获取第j个点
    Point getPoint(int j) const;
    // 获取第j个点的切向量
    Point getTangent(int j) const;
};
```

Moreover, I defined the BezierCurve class, which uses some functions in the curve class. In the BezierCurve class, I have designed the following features: calculating the value of the Bezier curve at a certain point, as well as fitting according to Algorithm 2.74.

```
class BezierCurve{
protected:
    std::vector<Point> control_points;
public:
    BezierCurve(const std::vector<Point>& points) : control_points(points) {}
    Point calculatePoint(double t) ;
    void approximateCurveWithBezier(const Curve& curve, int m, int num_points_per_segment, const
std::string& filename) ;
};
```

Below is the implementation of the function approximateCurveWithBezier and calculatePoint.

```
Point BezierCurve::calculatePoint(double t) {
    double x = std::pow(1 - t, 3) * control_points[0].x +
```

```

        3 * t * std::pow(1 - t, 2) * control_points[1].x +
        3 * std::pow(t, 2) * (1 - t) * control_points[2].x +
        std::pow(t, 3) * control_points[3].x;
double y = std::pow(1 - t, 3) * control_points[0].y +
        3 * t * std::pow(1 - t, 2) * control_points[1].y +
        3 * std::pow(t, 2) * (1 - t) * control_points[2].y +
        std::pow(t, 3) * control_points[3].y;
return Point(x, y);
}

void BezierCurve::approximateCurveWithBezier(const Curve& curve, int m, int num_points_per_segment,
const std::string& filename) {
    std::ofstream file(filename);

    for (int j = 0; j < m; ++j) {
        Point p_j = curve.getPoint(j);
        Point p_j1 = curve.getPoint(j + 1);
        Point tangent = curve.getTangent(j);

        Point q_0 = p_j;
        Point q_1 = Point(p_j.x + tangent.x / 3.0, p_j.y + tangent.y / 3.0);
        Point q_2 = Point(p_j1.x - tangent.x / 3.0, p_j1.y - tangent.y / 3.0);
        Point q_3 = p_j1;

        BezierCurve bezier({q_0, q_1, q_2, q_3});

        for (int i = 0; i <= num_points_per_segment; ++i) {
            double t = static_cast<double>(i) / num_points_per_segment;
            Point p = bezier.calculatePoint(t);
            file << p.x << " " << p.y << std::endl;
        }
    }

    file.close();
}

```

Result

The result is shown in the following figure. [\[3\]](#)

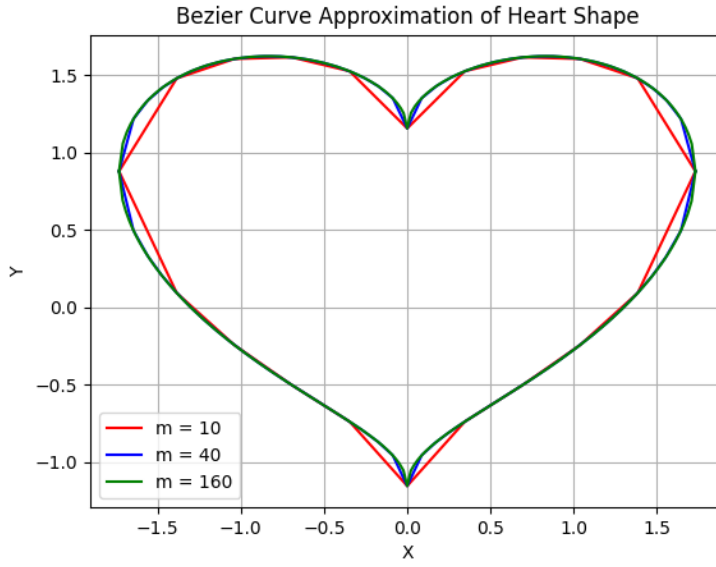


Figure 6: Bezier curve

Acknowledgement

References

- [1] DeepSeek AI. DeepSeek. <https://www.deepseek.com/>. Accessed: 2024-09-29.
- [2] Moonshot AI. Kimi Assistant. <https://kimi.moonshot.cn/>. Accessed: 2024-09-29.
- [3] OpenAI. ChatGPT: A Conversational Agent based on GPT-4 Architecture. <https://chat.openai.com>. Accessed: 2024-09-29.